Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3

По дисциплине: «АПО ЭВМиС»

Тема: «Изучение аппаратно-программной архитектуры процессоров семейства Intel - макроопределения»

Выполнил:
Студент 4-го курса
Группы АС-66
Ярома А. И.
Проверил:
Кулик А. Д.

Брест 2026

Цель работы

Изучить приёмы разработки параметризованных макроопределений и их применения в программах на ассемблере.

Задание. Вариант 12

Задана строка слов. Словом считается последовательность символов, разделённая пробелами. Написать макроопределение, удаляющее из исходной строки слова, содержащие хотя бы один русский символ. Проверить работоспособность разработанного макроопределения.

Описание реализации

REMOVE_RUSSIAN_WORDS определено в remove_russian_words.inc между директивами MACRO и ENDM. Четыре формальных параметра задают адрес Unicode-буфера, исходную длину, переменную новой длины и переменную количества удалённых слов. При вызове MASM подставляет тело макроса непосредственно в программу. Обработка не подменяется вызовом процедуры.

Метки объявлены через LOCAL, поэтому повторные раскрытия не конфликтуют. Входы могут передаваться через регистры или переменные, длина также может быть константой. Выходы - отдельные именованные DWORD-переменные; выходная длина может совпадать со входной. ESP-relative операнды не поддерживаются.

ESI просматривает исходный буфер, EDI указывает позицию записи. Для каждого слова проверяются А-Я, а-я и отдельно Ё/ё. Если русская буква найдена, пропускается всё слово. Иначе оно копируется в уплотняемую строку. Пробелы сохраняются в исходном количестве: нормализация относится к варианту 7. Сложность O(n), дополнительная память макроса O(1).

Программа целиком написана на MASM x86 и использует Win32 API. Строка вводится до Enter; точка - обычный символ. Память входного буфера увеличивается по мере необходимости. Один элемент DWORD хранит один символ Unicode. Искусственное ограничение в 100 символов из ЛР2 не применяется.

Текст программы

Макроопределение обработки строки: remove_russian_words.inc

```
; Lab 3, variant 12. One character is one Unicode DWORD.
; A word is a maximal sequence of characters other than ASCII space (20h).
; Delete a whole word if it contains А-Я, а-я, Ё or ё. Preserve all spaces.
;
; pText: pointer value (register or DWORD memory); inLength: character count.
; outLength and removedWords: distinct named DWORD variables (static or
; EBP-relative locals), outside the text buffer. outLength may alias inLength.
; Operands must not be ESP-relative. All GPRs are preserved; arithmetic flags
; are unspecified. Buffer must have room for inLength + 1 DWORDs.
; Time O(n), additional workspace O(1). No procedure calls inside this macro.
REMOVE_RUSSIAN_WORDS MACRO pText:REQ, inLength:REQ, outLength:REQ, removedWords:REQ
    LOCAL next_item, copy_space, scan_word, check_yo, russian_found, scan_advance
    LOCAL word_end, remove_word, copy_word, finished
    pushad
    push inLength
    push pText
    pop esi
    pop ecx
    mov edi, esi
    lea edx, [esi + ecx * 4]
    sub esp, 8
    mov DWORD PTR [esp], 0       ; number of removed words
    mov [esp + 4], esi           ; original buffer address
next_item:
    cmp esi, edx
    jae finished
    mov eax, [esi]
    cmp eax, 20h
    je copy_space
    mov ebx, esi                 ; beginning of the word
    xor ecx, ecx                 ; Russian-letter flag
scan_word:
    cmp esi, edx
    jae word_end
    mov eax, [esi]
    cmp eax, 20h
    je word_end
    cmp eax, 0410h
    jb check_yo
    cmp eax, 044Fh
    jbe russian_found
check_yo:
    cmp eax, 0401h
    je russian_found
    cmp eax, 0451h
    je russian_found
    jmp scan_advance
russian_found:
    mov ecx, 1
scan_advance:
    add esi, 4
    jmp scan_word
word_end:
    test ecx, ecx
    jnz remove_word
copy_word:
    cmp ebx, esi
    jae next_item
    mov eax, [ebx]
    mov [edi], eax
    add ebx, 4
    add edi, 4
    jmp copy_word
remove_word:
    inc DWORD PTR [esp]
    jmp next_item
copy_space:
    mov [edi], eax
```

```
72      add esi, 4
73      add edi, 4
74      jmp next_item
75  finished:
76      mov DWORD PTR [edi], 0
77      mov eax, edi
78      sub eax, [esp + 4]
79      shr eax, 2
80      mov outLength, eax
81      mov eax, [esp]
82      mov removedWords, eax
83      add esp, 8
84      popad
85  ENDM
```

Объявления интерфейса ввода и вывода: unicode_io.inc

```
 1  ; Shared declarations for the assembly-only console/file I/O module.
 2  CHAR_EOF EQU -1
 3  CHAR_INVALID EQU -2
 5  InputContext STRUCT
 6      handle DWORD ?
 7      console DWORD ?
 8      mode DWORD ?
 9  InputContext ENDS
11  ReadCharacter PROTO :PTR InputContext
12  WriteMessage PROTO :DWORD, :DWORD
13  PrintText PROTO :DWORD, :PTR DWORD, :DWORD
14  PrintUnsigned PROTO :DWORD, :DWORD
```

Ввод строки, макрокоманда и вывод результата: main.asm

```
 1  ; Lab 3, variant 12. Entire application is MASM x86, without C/C++ runtime.
 2  .386
 3  .model flat, stdcall
 4  option casemap:none
 5  include unicode_io.inc
 6  include remove_russian_words.inc
 8  INITIAL_CAPACITY EQU 256
 9  LINE_OK EQU 0
10  LINE_BAD_ENCODING EQU 1
11  LINE_NO_MEMORY EQU 2
13  GetStdHandle PROTO :DWORD
14  GetConsoleMode PROTO :DWORD, :PTR DWORD
15  SetConsoleOutputCP PROTO :DWORD
16  GetProcessHeap PROTO
17  HeapAlloc PROTO :DWORD, :DWORD, :DWORD
18  HeapReAlloc PROTO :DWORD, :DWORD, :DWORD, :DWORD
19  HeapFree PROTO :DWORD, :DWORD, :DWORD
20  ExitProcess PROTO :DWORD
22  .const
23  msgHeader BYTE 'АПО ЭВМиС. Лабораторная работа №3',13,10
24      BYTE 'Ярома А. И. | АС-66 | Вариант 12 | MASM x86',13,10
25      BYTE 'Макрос удаляет слова с русскими буквами А-Я, а-я, Ё, ё.',13,10
26      BYTE 'Разделитель слов - пробел. Исходные пробелы сохраняются.',13,10
27      BYTE 'Введите строку и нажмите Enter (точка - обычный символ):',13,10,0
28  msgOriginal BYTE 13,10,'Исходная строка: "',0
29  msgResult BYTE '"',13,10,'Результат: "',0
30  msgLength BYTE '"',13,10,'Длина результата: ',0
31  msgRemoved BYTE 13,10,'Удалено слов: ',0
32  msgEnd BYTE 13,10,0
33  msgEncoding BYTE 'Ошибка: некорректная кодировка Unicode. Обработка не выполнена.',13,10,0
34  msgMemory BYTE 'Ошибка: недостаточно памяти для строки. Обработка не выполнена.',13,10,0
36  .code
38  ; Read a line until Enter or EOF. Dots are ordinary characters.
39  ; Grow the input buffer as needed; no inherited 100-character limit.
40  ReadLine PROC USES esi edi ebx, pContext:PTR InputContext, hHeap:DWORD,
41      ppText:PTR DWORD, pLength:PTR DWORD
42      LOCAL capacity:DWORD, character:DWORD
43      mov eax, ppText
44      mov DWORD PTR [eax], 0
45      mov eax, pLength
```

```
 46      mov DWORD PTR [eax], 0
 47      mov capacity, INITIAL_CAPACITY
 48      invoke HeapAlloc, hHeap, 0, INITIAL_CAPACITY * 4
 49      mov esi, eax
 50      test esi, esi
 51      jz line_no_memory
 52      xor edi, edi
 53  line_next:
 54      invoke ReadCharacter, pContext
 55      cmp eax, CHAR_EOF
 56      je line_finished
 57      cmp eax, CHAR_INVALID
 58      je line_bad_encoding
 59      cmp eax, 13
 60      je line_finished
 61      cmp eax, 10
 62      je line_finished
 63      mov character, eax
 64      lea edx, [edi + 1]
 65      cmp edx, capacity
 66      jb line_store
 67      mov eax, capacity
 68      cmp eax, 10000000h
 69      jae line_no_memory
 70      shl eax, 1
 71      mov capacity, eax
 72      shl eax, 2
 73      invoke HeapReAlloc, hHeap, 0, esi, eax
 74      test eax, eax
 75      jz line_no_memory
 76      mov esi, eax
 77  line_store:
 78      mov eax, character
 79      mov [esi + edi * 4], eax
 80      inc edi
 81      jmp line_next
 82  line_finished:
 83      mov DWORD PTR [esi + edi * 4], 0
 84      mov eax, ppText
 85      mov [eax], esi
 86      mov eax, pLength
 87      mov [eax], edi
 88      mov eax, LINE_OK
 89      ret
 90  line_bad_encoding:
 91      mov ebx, LINE_BAD_ENCODING
 92      jmp line_cleanup
 93  line_no_memory:
 94      mov ebx, LINE_NO_MEMORY
 95  line_cleanup:
 96      test esi, esi
 97      jz line_return_error
 98      invoke HeapFree, hHeap, 0, esi
 99  line_return_error:
100      mov eax, ebx
101      ret
102  ReadLine ENDP
104  ProgramMain PROC
105      LOCAL context:InputContext
106      LOCAL hOutput:DWORD, hHeap:DWORD, textPointer:DWORD
107      LOCAL charCount:DWORD, removedCount:DWORD, status:DWORD
108      invoke GetStdHandle, -10
109      mov context.handle, eax
110      invoke GetConsoleMode, context.handle, ADDR context.mode
111      mov context.console, eax
112      invoke GetStdHandle, -11
113      mov hOutput, eax
114      invoke SetConsoleOutputCP, 65001
115      invoke GetProcessHeap
116      mov hHeap, eax
117      invoke WriteMessage, hOutput, ADDR msgHeader
118      invoke ReadLine, ADDR context, hHeap, ADDR textPointer, ADDR charCount
119      cmp eax, LINE_OK
120      jne main_error
```

```
121      invoke WriteMessage, hOutput, ADDR msgOriginal
122      invoke PrintText, hOutput, textPointer, charCount
124      ; This is a macro invocation, expanded by MASM here, not a CALL.
125      REMOVE_RUSSIAN_WORDS textPointer, charCount, charCount, removedCount
127      invoke WriteMessage, hOutput, ADDR msgResult
128      invoke PrintText, hOutput, textPointer, charCount
129      invoke WriteMessage, hOutput, ADDR msgLength
130      invoke PrintUnsigned, hOutput, charCount
131      invoke WriteMessage, hOutput, ADDR msgRemoved
132      invoke PrintUnsigned, hOutput, removedCount
133      invoke WriteMessage, hOutput, ADDR msgEnd
134      invoke HeapFree, hHeap, 0, textPointer
135      xor eax, eax
136      ret
137  main_error:
138      mov status, eax
139      mov edx, OFFSET msgMemory
140      cmp status, LINE_BAD_ENCODING
141      jne main_print_error
142      mov edx, OFFSET msgEncoding
143  main_print_error:
144      invoke WriteMessage, hOutput, edx
145      mov eax, 1
146      ret
147  ProgramMain ENDP
149  start:
150      invoke ProgramMain
151      invoke ExitProcess, eax
152  END start
```

Ассемблерные средства Unicode-ввода/вывода: unicode_io.asm

```
 1  ; Shared Unicode I/O, MASM x86. No text-processing algorithm in this module.
 2  .386
 3  .model flat, stdcall
 4  option casemap:none
 5  include unicode_io.inc
 7  ReadFile PROTO :DWORD, :DWORD, :DWORD, :PTR DWORD, :DWORD
 8  ReadConsoleW PROTO :DWORD, :DWORD, :DWORD, :PTR DWORD, :DWORD
 9  WriteFile PROTO :DWORD, :DWORD, :DWORD, :PTR DWORD, :DWORD
11  PUBLIC ReadCharacter, WriteMessage, PrintText, PrintUnsigned
12  .code
13  ReadRaw PROC USES esi, pContext:PTR InputContext
14      LOCAL unitValue:DWORD, received:DWORD
15      mov esi, pContext
16      mov unitValue, 0
17      cmp [esi].InputContext.console, 0
18      je raw_file
19      invoke ReadConsoleW, [esi].InputContext.handle, ADDR unitValue, 1, ADDR received, 0
20      jmp raw_done
21  raw_file:
22      invoke ReadFile, [esi].InputContext.handle, ADDR unitValue, 1, ADDR received, 0
23  raw_done:
24      test eax, eax
25      jz raw_eof
26      cmp received, 0
27      je raw_eof
28      mov eax, unitValue
29      ret
30  raw_eof:
31      mov eax, CHAR_EOF
32      ret
33  ReadRaw ENDP
35  ; Decode one Unicode scalar. Console input is UTF-16; redirected input is UTF-8.
36  ; EAX = scalar, CHAR_EOF, or CHAR_INVALID. No text-sized temporary buffer.
37  ReadCharacter PROC USES esi ebx edi, pContext:PTR InputContext
38      LOCAL minimum:DWORD
39      mov esi, pContext
40      invoke ReadRaw, esi
41      cmp eax, CHAR_EOF
42      je character_return
43      cmp [esi].InputContext.console, 0
44      je decode_utf8
```

```
 45      cmp eax, 0D800h
 46      jb character_return
 47      cmp eax, 0DBFFh
 48      ja check_low_surrogate
 49      sub eax, 0D800h
 50      shl eax, 10
 51      mov ebx, eax
 52      invoke ReadRaw, esi
 53      cmp eax, 0DC00h
 54      jb character_invalid
 55      cmp eax, 0DFFFh
 56      ja character_invalid
 57      sub eax, 0DC00h
 58      lea eax, [eax + ebx + 10000h]
 59      ret
 60  check_low_surrogate:
 61      cmp eax, 0DFFFh
 62      jbe character_invalid
 63      ret
 64  decode_utf8:
 65      cmp eax, 80h
 66      jb character_return
 67      mov minimum, 80h
 68      mov edi, 1
 69      cmp eax, 0C2h
 70      jb character_invalid
 71      cmp eax, 0DFh
 72      jbe utf8_two
 73      mov minimum, 800h
 74      mov edi, 2
 75      cmp eax, 0EFh
 76      jbe utf8_three
 77      cmp eax, 0F4h
 78      ja character_invalid
 79      mov minimum, 10000h
 80      mov edi, 3
 81      and eax, 7
 82      jmp utf8_continue
 83  utf8_three:
 84      and eax, 0Fh
 85      jmp utf8_continue
 86  utf8_two:
 87      and eax, 1Fh
 88  utf8_continue:
 89      mov ebx, eax
 90  utf8_next:
 91      invoke ReadRaw, esi
 92      mov edx, eax
 93      and edx, 0FFFFFFC0h
 94      cmp edx, 80h
 95      jne character_invalid
 96      shl ebx, 6
 97      and eax, 3Fh
 98      or ebx, eax
 99      dec edi
100      jnz utf8_next
101      mov eax, ebx
102      cmp eax, minimum
103      jb character_invalid
104      cmp eax, 10FFFFh
105      ja character_invalid
106      cmp eax, 0D800h
107      jb character_return
108      cmp eax, 0DFFFh
109      jbe character_invalid
110  character_return:
111      ret
112  character_invalid:
113      mov eax, CHAR_INVALID
114      ret
115  ReadCharacter ENDP
117  WriteAll PROC USES esi ebx, hOutput:DWORD, pBytes:DWORD, byteCount:DWORD
118      LOCAL written:DWORD
119      mov esi, pBytes
```

```
120      mov ebx, byteCount
121  write_more:
122      test ebx, ebx
123      jz write_success
124      invoke WriteFile, hOutput, esi, ebx, ADDR written, 0
125      test eax, eax
126      jz write_return
127      cmp written, 0
128      je write_failed
129      add esi, written
130      sub ebx, written
131      jmp write_more
132  write_failed:
133      xor eax, eax
134  write_return:
135      ret
136  write_success:
137      mov eax, 1
138      ret
139  WriteAll ENDP
141  WriteMessage PROC hOutput:DWORD, pMessage:DWORD
142      mov edx, pMessage
143      xor ecx, ecx
144  message_length:
145      cmp BYTE PTR [edx + ecx], 0
146      je message_write
147      inc ecx
148      jmp message_length
149  message_write:
150      invoke WriteAll, hOutput, edx, ecx
151      ret
152  WriteMessage ENDP
154  ; Encode one scalar in a fixed four-byte local, then print it.
155  PrintCharacter PROC USES ebx, hOutput:DWORD, value:DWORD
156      LOCAL encoded:DWORD, byteCount:DWORD
157      mov eax, value
158      lea edx, encoded
159      mov byteCount, 1
160      cmp eax, 80h
161      jb encode_last
162      mov byteCount, 2
163      cmp eax, 800h
164      jb encode_continuations
165      mov byteCount, 3
166      cmp eax, 10000h
167      jb encode_continuations
168      mov byteCount, 4
169  encode_continuations:
170      mov ecx, byteCount
171      dec ecx
172  encode_tail:
173      mov ebx, eax
174      and ebx, 3Fh
175      or bl, 80h
176      mov [edx + ecx], bl
177      shr eax, 6
178      dec ecx
179      jnz encode_tail
180      cmp byteCount, 2
181      je encode_head_two
182      cmp byteCount, 3
183      je encode_head_three
184      or eax, 0F0h
185      jmp encode_store_head
186  encode_head_three:
187      or eax, 0E0h
188      jmp encode_store_head
189  encode_head_two:
190      or eax, 0C0h
191  encode_store_head:
192      mov [edx], al
193      jmp encode_write
194  encode_last:
195      mov [edx], al
```

```
196 encode_write:
197     invoke WriteAll, hOutput, ADDR encoded, byteCount
198     ret
199 PrintCharacter ENDP
201 ; Stage 4: print the transformed text, without changing the working buffer.
202 PrintText PROC USES esi ebx, hOutput:DWORD, pText:PTR DWORD, charCount:DWORD
203     mov esi, pText
204     mov ebx, charCount
205 print_next:
206     test ebx, ebx
207     jz print_done
208     invoke PrintCharacter, hOutput, DWORD PTR [esi]
209     add esi, 4
210     dec ebx
211     jmp print_next
212 print_done:
213     ret
214 PrintText ENDP
216 PrintUnsigned PROC USES ebx edi, hOutput:DWORD, value:DWORD
217     LOCAL digits[10]:BYTE
218     lea edi, digits[10]
219     mov eax, value
220     mov ebx, 10
221 number_next:
222     xor edx, edx
223     div ebx
224     add dl, '0'
225     dec edi
226     mov [edi], dl
227     test eax, eax
228     jnz number_next
229     lea ecx, digits[10]
230     sub ecx, edi
231     invoke WriteAll, hOutput, edi, ecx
232     ret
233 PrintUnsigned ENDP
234 END
```

Результаты выполнения

Скриншоты получены при запуске собранной программы Release / x86. Ввод передан из файлов UTF-8. Исходная строка, результат и число удалённых слов напечатаны самой программой.

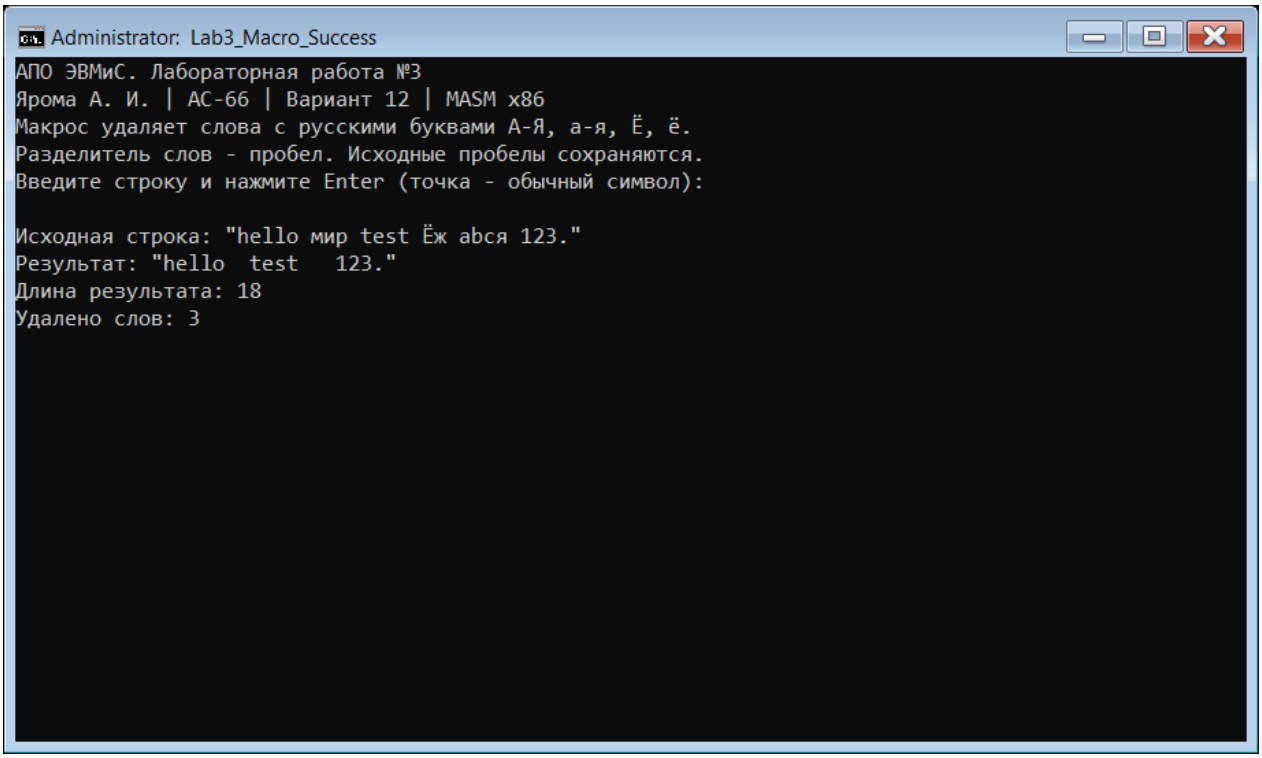


Рисунок 1 - Удалены «мир», «Ёж» и смешанное слово «abcя»; исходные пробелы и точка сохранены.

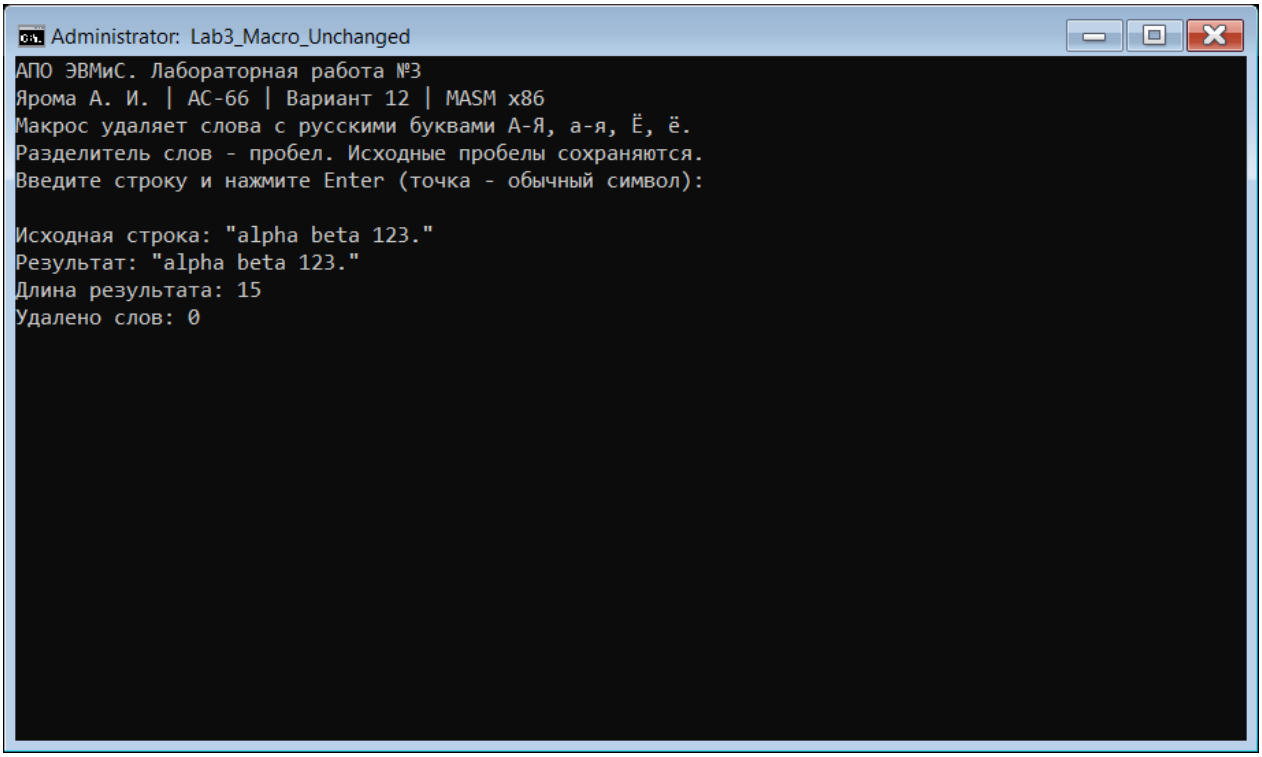


Рисунок 2 - Русских букв нет: строка «alpha beta 123.» не изменилась.

Проверка макроопределения

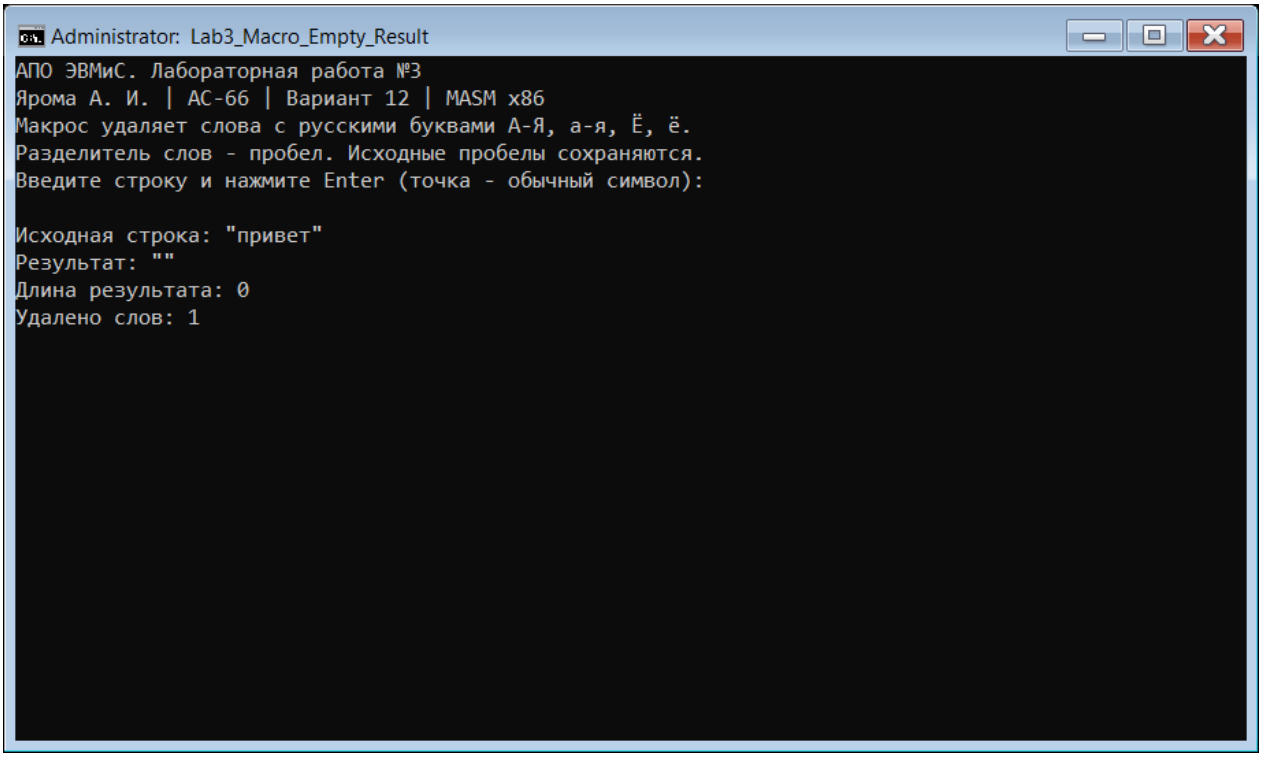


Рисунок 3 - Единственное русское слово удалено; результат - пустая строка.

Случай	Ожидаемое поведение
hello мир test	hello [два пробела] test: разделители сохранены
mixЯabc keep ёж 123.	Удаляются целиком два слова с русскими буквами
Ё, ё и А-Я, а-я	Слово с любой русской буквой удаляется
Латиница, I/i, другие языки	Без русской буквы удаление не выполняется
Пустая строка / пробелы	Пустой результат / исходные пробелы
Длинные строки	Буфер увеличивается; лимит 100 не применяется

Проверено 3348 случаев непосредственно для макроса. В каждом он раскрывается дважды в одной процедуре: проверяются LOCAL-метки, входы в регистрах и памяти, совпадение входной и выходной длины, сохранность регистров и стека, результат, счётчик и границы буфера. Дополнительно прошли по 314 запусков Debug и Release и по 8 тестов настоящего ReadConsoleW. Всего 3992 сценария; макрос испытан на длинах до 100000, программа - до 65536 символов.

Вывод

Разработано и проверено параметризованное макроопределение варианта 12. Оно удаляет целиком слова с русскими буквами и сохраняет остальные символы и пробелы. Освоены MACRO, ENDM, формальные и фактические параметры, LOCAL и повторное макрорасширение. Проверки подтвердили корректность обработки и возможность безопасного повторного применения.